

NOVA

Mobile Application Implementation Guide

Flutter (Dart) | Android (API 26+) | Version 1.0

Field	Value
Framework	Flutter 3.22+ (Dart 3.4+)
Target Platform	Android API 26 (Android 8.0) to API 35 (Android 15)
Architecture Pattern	Clean Architecture + BLoC (Business Logic Component)
State Management	flutter_bloc 8.x
Local Database	drift (SQLite ORM for Dart)
ML Runtime	tflite_flutter 0.10+
OCR	google_mlkit_text_recognition 0.13+
Camera	camera 0.10+
TTS	flutter_tts 4.x
Speech Recognition	speech_to_text 6.x
HTTP Client	dio 5.x
Dependency Injection	get_it + injectable
Document Version	1.0 June 2026

1. Architecture Overview

1.1 Clean Architecture with BLoC

NOVA's Flutter codebase follows Clean Architecture principles, separating the application into three concentric layers: Presentation, Domain, and Data. State management is handled by the BLoC (Business Logic Component) pattern, which keeps UI widgets reactive and testable.

- **Presentation Layer:** Flutter widgets, BLoC classes, and UI state. Widgets observe BLoC states and dispatch events. No business logic lives here.
- **Domain Layer:** Use cases (interactors), entity classes, and repository interfaces. Pure Dart, zero Flutter dependency. This is the application's business logic.
- **Data Layer:** Repository implementations, data sources (remote API, local SQLite, TFLite models, camera stream), and data transfer objects (DTOs).

The BLoC pattern replaces traditional callbacks with streams, making the asynchronous nature of camera processing, TTS, and network calls predictable and testable. Each feature module (obstacle detection, OCR, scene description, currency, face recognition) has its own BLoC.

1.2 Feature-First Folder Structure

```

lib/
├── main.dart # App entry point
├── injection_container.dart # GetIt dependency injection setup
├── core/
│   └── constants/ # App-wide constants (thresholds, URLs, model
paths)
├── error/ # Failure classes and error handling
├── network/ # Dio HTTP client, connectivity checker
├── tts/ # TTS service wrapper
├── audio/ # Audio session management
├── permissions/ # Permission handler service
├── usecases/ # Base UseCase abstract class
├── features/
│   └── auth/ # Login, registration, JWT management
│       ├── data/
│       │   ├── datasources/ # remote_auth_datasource.dart
│       │   ├── models/ # user_model.dart
│       │   └── repositories/ # auth_repository_impl.dart
│       ├── domain/
│       │   ├── entities/ # user.dart
│       │   ├── repositories/ # auth_repository.dart (abstract)
│       │   └── usecases/ # login_usecase.dart, register_usecase.dart
│       └── presentation/
│           └── bloc/ # auth_bloc.dart, auth_event.dart,
auth state.dart
├── pages/ # login_page.dart, onboarding_page.dart
├── obstacle_detection/
│   ├── data/
│   │   ├── datasources/ # tflite_obstacle_datasource.dart
│   │   ├── models/ # detection_result_model.dart
│   │   └── repositories/ # obstacle_repo_impl.dart
│   ├── domain/
│   │   ├── entities/ # obstacle_detection_result.dart
│   │   ├── repositories/ # obstacle_repository.dart
│   │   └── usecases/ # detect_obstacles_usecase.dart
│   └── presentation/
│       ├── bloc/ # obstacle_bloc.dart
│       └── pages/ # obstacle_page.dart
├── ocr/ # Same structure as obstacle_detection
├── scene_description/ # Same structure
├── currency_detection/ # Same structure
├── face_recognition/ # Same structure
assets/

```

```
models/                                # Bundled TFLite model files (.tflite)
├── obstacle_detection_v1.tflite
├── currency_detection_v1.tflite
├── face_detection_blazeface.tflite
├── face_embedding_mobilefacenet.tflite
labels/
├── coco_labels.txt
└── cfa_labels.txt
```

2. Project Setup and Dependencies

2.1 pubspec.yaml

```
name: nova_assistive
description: NOVA - Navigational Object and Voice Assistant
version: 1.0.0+1

environment:
  sdk: '>=3.4.0 <4.0.0'
  flutter: '>=3.22.0'

dependencies:
  flutter:
    sdk: flutter

  # State management
  flutter_bloc: ^8.1.6
  bloc: ^8.1.4
  equatable: ^2.0.5

  # Dependency injection
  get_it: ^7.7.0
  injectable: ^2.4.4

  # Camera & vision
  camera: ^0.10.6
  google_mlkit_text_recognition: ^0.13.0
  tflite_flutter: ^0.10.4
  tflite_flutter_helper: ^0.3.1
  image: ^4.2.0

  # TTS and Speech
  flutter_tts: ^4.0.2
  speech_to_text: ^6.6.2
  audio_session: ^0.1.21

  # Networking
  dio: ^5.4.3+1
  connectivity_plus: ^6.0.3

  # Local storage
  drift: ^2.18.0
  sqlite3_flutter_libs: ^0.5.22
  path_provider: ^2.1.3
  path: ^1.9.0
  shared_preferences: ^2.2.3
  flutter_secure_storage: ^9.0.0

  # Location & permissions
  geolocator: ^12.0.0
  permission_handler: ^11.3.1
  vibration: ^2.0.0

  # Utilities
  dartz: ^0.10.1
  rxdart: ^0.27.7
  logger: ^2.4.0
  uuid: ^4.4.2
  intl: ^0.19.0

dev_dependencies:
  flutter_test:
    sdk: flutter
```

```
build_runner: ^2.4.11
drift_dev: ^2.18.0
injectable_generator: ^2.6.2
bloc_test: ^9.1.7
mocktail: ^1.0.4
flutter_lints: ^4.0.0

flutter:
  assets:
    - assets/models/
    - assets/labels/
```

2.2 Android Manifest Permissions

```
<!-- AndroidManifest.xml -->
<uses-permission android:name='android.permission.CAMERA' />
<uses-permission android:name='android.permission.RECORD_AUDIO' />
<uses-permission android:name='android.permission.ACCESS_FINE_LOCATION' />
<uses-permission android:name='android.permission.ACCESS_COARSE_LOCATION' />
<uses-permission android:name='android.permission.INTERNET' />
<uses-permission android:name='android.permission.VIBRATE' />
<uses-permission android:name='android.permission.SEND_SMS' />
<uses-permission android:name='android.permission.FOREGROUND_SERVICE' />
<uses-permission android:name='android.permission.WAKE_LOCK' />

<!-- Required for TFLite GPU delegate (optional on low-end devices) -->
<uses-feature android:name='android.hardware.camera' />
<uses-feature android:name='android.hardware.camera.autofocus' />
```

3. Core Services

3.1 TTS Service

The TTS service wraps Flutter's `flutter_tts` plugin with a priority queue, so safety-critical obstacle alerts always interrupt lower-priority announcements (scene descriptions, OCR output).

```
// lib/core/tts/tts_service.dart
import 'package:flutter_tts/flutter_tts.dart';
import 'package:injectable/injectable.dart';

enum TtsPriority { critical, high, normal, low }

@singleton
class TtsService {
  final FlutterTts _tts = FlutterTts();
  TtsPriority? _currentPriority;

  Future<void> init(String languageCode) async {
    await _tts.setLanguage(languageCode); // 'en-CM' or 'fr-CM'
    await _tts.setSpeechRate(1.0);
    await _tts.setVolume(1.0);
    await _tts.setPitch(1.0);
    _tts.setCompletionHandler(() => _currentPriority = null);
  }

  Future<void> speak(
    String text, {
    TtsPriority priority = TtsPriority.normal,
    bool interrupt = false,
  }) async {
    // Critical and high-priority alerts always interrupt
    final shouldInterrupt = interrupt ||
      priority == TtsPriority.critical ||
      (priority == TtsPriority.high &&
        _currentPriority != TtsPriority.critical);

    if (shouldInterrupt) await _tts.stop();

    _currentPriority = priority;
    await _tts.speak(text);
  }

  Future<void> stop() async => await _tts.stop();

  Future<void> setSpeechRate(double rate) async {
    await _tts.setSpeechRate(rate.clamp(0.5, 2.0));
  }

  Future<void> setLanguage(String code) async {
    await _tts.setLanguage(code);
  }
}
```

3.2 Connectivity Service

```
// lib/core/network/connectivity_service.dart
import 'package:connectivity_plus/connectivity_plus.dart';
import 'package:injectable/injectable.dart';

@singleton
class ConnectivityService {
  final Connectivity _connectivity = Connectivity();

  Stream<bool> get onConnectivityChanged =>
```

```

        _connectivity.onConnectivityChanged.map(
          (results) => results.any((r) => r != ConnectivityResult.none),
        );

Future<bool> get isConnected async {
  final results = await _connectivity.checkConnectivity();
  return results.any((r) => r != ConnectivityResult.none);
}
}

```

3.3 Dio HTTP Client with JWT Interceptor

```

// lib/core/network/dio_client.dart
import 'package:dio/dio.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:injectable/injectable.dart';

@singleton
class DioClient {
  late final Dio _dio;
  final FlutterSecureStorage _storage;

  DioClient(this._storage) {
    _dio = Dio(BaseOptions(
      baseUrl: const String.fromEnvironment('API_BASE_URL',
        defaultValue: 'https://api.nova-assistive.cm'),
      connectTimeout: const Duration(seconds: 10),
      receiveTimeout: const Duration(seconds: 15),
      headers: {'Content-Type': 'application/json'},
    ));
    _dio.interceptors.add(_JwtInterceptor(_storage, _dio));
  }

  Dio get client => _dio;
}

class _JwtInterceptor extends QueuedInterceptor {
  final FlutterSecureStorage _storage;
  final Dio _dio;

  _JwtInterceptor(this._storage, this._dio);

  @override
  Future<void> onRequest(
    RequestOptions options,
    RequestInterceptorHandler handler,
  ) async {
    final token = await _storage.read(key: 'access_token');
    if (token != null) {
      options.headers['Authorization'] = 'Bearer \${token}';
    }
    handler.next(options);
  }

  @override
  Future<void> onError(
    DioException err,
    ErrorInterceptorHandler handler,
  ) async {
    if (err.response?.statusCode == 401) {
      // Attempt token refresh
      final refreshed = await _refreshToken();
      if (refreshed) {
        handler.resolve(await _dio.fetch(err.requestOptions));
        return;
      }
    }
  }
}

```

```
    }  
    handler.next(err);  
  }  
  
  Future<bool> _refreshToken() async {  
    try {  
      final refresh = await _storage.read(key: 'refresh_token');  
      if (refresh == null) return false;  
      final res = await _dio.post('/auth/refresh',  
        data: {'refresh_token': refresh});  
      await _storage.write(  
        key: 'access_token', value: res.data['access_token']);  
      return true;  
    } catch (_) {  
      return false;  
    }  
  }  
}
```


4. Feature Implementation: Obstacle Detection

4.1 Domain Layer

```
//
lib/features/obstacle_detection/domain/entities/obstacle_detection_result.dart
import 'package:equatable/equatable.dart';

enum ObstacleZone { near, warning, clear }
enum ObstacleDirection { left, center, right }

class DetectedObstacle extends Equatable {
  final String label;
  final double confidence;
  final ObstacleZone zone;
  final ObstacleDirection direction;
  final double estimatedDistanceMeters;
  final String trackingId;

  const DetectedObstacle({
    required this.label,
    required this.confidence,
    required this.zone,
    required this.direction,
    required this.estimatedDistanceMeters,
    required this.trackingId,
  });

  @override
  List<Object> get props => [trackingId, label, zone];
}
```

```
// lib/features/obstacle_detection/domain/usecases/detect_obstacles_usecase.dart
import 'package:dartz/dartz.dart';
import '../repositories/obstacle_repository.dart';
import '../entities/obstacle_detection_result.dart';
import '../../../core/error/failures.dart';

class DetectObstaclesUseCase {
  final ObstacleRepository repository;
  DetectObstaclesUseCase(this.repository);

  Stream<Either<Failure, List<DetectedObstacle>>> call(
    Stream<CameraImageData> cameraStream,
  ) {
    return repository.detectObstacles(cameraStream);
  }
}
```

4.2 Data Layer TFLite Inference

```
//
lib/features/obstacle_detection/data/datasources/tflite_obstacle_datasource.dart
import 'dart:isolate';
import 'package:tflite_flutter/tflite_flutter.dart';
import 'package:image/image.dart' as img;

class TfliteObstacleDatasource {
  late Interpreter _interpreter;
  late List<String> _labels;
  static const int _inputSize = 320; // SSD MobileNetV3 input
  static const double _nearThresholdMeters = 1.5;
  static const double _warningThresholdMeters = 3.0;
  static const double _confThreshold = 0.55;
}
```

```

// Reference widths in meters for distance estimation per class
static const Map<String, double> _referenceWidths = {
  'person': 0.5, 'car': 1.8, 'motorcycle': 0.8,
  'bicycle': 0.6, 'bus': 2.5, 'truck': 2.4,
  'chair': 0.5, 'table': 1.2, 'door': 0.9,
};

Future<void> init() async {
  final options = InterpreterOptions()..threads = 2;
  // Use NNAPI delegate if available (NPU acceleration)
  try {
    options.addDelegate(NnApiDelegate());
  } catch (_) {
    // Fall back to CPU acceptable on low-end devices
  }
  _interpreter = await Interpreter.fromAsset(
    'assets/models/obstacle_detection_v1.tflite',
    options: options,
  );
  final labelsData = await
rootBundle.loadString('assets/labels/coco_labels.txt');
  _labels = labelsData.split('\n');
}

List<RawDetection> runInference(img.Image frame) {
  // Resize and normalise to [0, 255] INT8
  final resized = img.copyResize(frame, width: _inputSize, height:
_inputSize);
  final input = _imageToByteListUint8(resized);

  // Output tensors for SSD: [boxes, classes, scores, count]
  final boxes = List.generate(1, (_) => List.generate(10, (_) =>
List.filled(4, 0.0)));
  final classes = List.generate(1, (_) => List.filled(10, 0.0));
  final scores = List.generate(1, (_) => List.filled(10, 0.0));
  final count = List.filled(1, 0.0);

  _interpreter.runForMultipleInputs([input], {
    0: boxes, 1: classes, 2: scores, 3: count,
  });

  final results = <RawDetection>[];
  final numDetections = count[0].toInt();
  for (int i = 0; i < numDetections; i++) {
    if (scores[0][i] < _confThreshold) continue;
    final classIdx = classes[0][i].toInt();
    results.add(RawDetection(
      label: _labels[classIdx],
      confidence: scores[0][i],
      boundingBox: boxes[0][i], // [top, left, bottom, right] normalised
    ));
  }
  return results;
}

double _estimateDistance(String label, double boxWidthNorm, int frameWidth) {
  final refWidth = _referenceWidths[label] ?? 0.5;
  // Pinhole camera model: distance ~ (focalLength * refWidth) / pixelWidth
  // focalLength estimated at 600px for typical 8MP phone camera
  const double focalLength = 600.0;
  final pixelWidth = boxWidthNorm * frameWidth;
  if (pixelWidth <= 0) return 999.0;
  return (focalLength * refWidth) / pixelWidth;
}

ObstacleZone _classifyZone(double distanceM) {

```

```

    if (distanceM <= _nearThresholdMeters) return ObstacleZone.near;
    if (distanceM <= _warningThresholdMeters) return ObstacleZone.warning;
    return ObstacleZone.clear;
  }

  ObstacleDirection _classifyDirection(double left, double right) {
    final center = (left + right) / 2;
    if (center < 0.33) return ObstacleDirection.left;
    if (center > 0.67) return ObstacleDirection.right;
    return ObstacleDirection.center;
  }

  Uint8List _imageToByteListUint8(img.Image image) {
    final bytes = Uint8List(1 * _inputSize * _inputSize * 3);
    int idx = 0;
    for (int y = 0; y < _inputSize; y++) {
      for (int x = 0; x < _inputSize; x++) {
        final pixel = image.getPixel(x, y);
        bytes[idx++] = img.getRed(pixel);
        bytes[idx++] = img.getGreen(pixel);
        bytes[idx++] = img.getBlue(pixel);
      }
    }
    return bytes;
  }
}

```

4.3 BLoC Layer

```

// lib/features/obstacle_detection/presentation/bloc/obstacle_bloc.dart
import 'dart:async';
import 'package:flutter_bloc/flutter_bloc.dart';
import '../domain/usecases/detect_obstacles_usecase.dart';
import '../core/tts/tts_service.dart';

// --- Events ---
abstract class ObstacleEvent {}
class StartObstacleDetection extends ObstacleEvent {}
class StopObstacleDetection extends ObstacleEvent {}
class _ObstaclesDetected extends ObstacleEvent {
  final List<DetectedObstacle> obstacles;
  _ObstaclesDetected(this.obstacles);
}

// --- States ---
abstract class ObstacleState {}
class ObstacleIdle extends ObstacleState {}
class ObstacleDetecting extends ObstacleState {
  final List<DetectedObstacle> obstacles;
  ObstacleDetecting(this.obstacles);
}
class ObstacleError extends ObstacleState { final String message;
ObstacleError(this.message); }

// --- BLoC ---
class ObstacleBloc extends Bloc<ObstacleEvent, ObstacleState> {
  final DetectObstaclesUseCase _detectUseCase;
  final TtsService _tts;
  StreamSubscription? _detectionSubscription;

  // Alert suppression: track last alert time per tracking ID
  final Map<String, DateTime> _lastAlertTime = {};
  static const _suppressionPeriod = Duration(seconds: 2);

  ObstacleBloc(this._detectUseCase, this._tts) : super(ObstacleIdle()) {
    on<StartObstacleDetection>(_onStart);
  }
}

```

```

    on<StopObstacleDetection>(_onStop);
    on<_ObstaclesDetected>(_onObstaclesDetected);
  }

  Future<void> _onStart(StartObstacleDetection event, Emitter<ObstacleState>
emit) async {
    await _tts.speak('Obstacle detection started.', priority: TtsPriority.high);
    _detectionSubscription = _detectUseCase(cameraStream).listen(
      (result) => result.fold(
        (failure) => add(ObstacleError(failure.message)),
        (obstacles) => add(_ObstaclesDetected(obstacles)),
      ),
    );
    emit(ObstacleDetecting([]));
  }

  Future<void> _onObstaclesDetected(
    _ObstaclesDetected event,
    Emitter<ObstacleState> emit,
  ) async {
    final now = DateTime.now();
    for (final obstacle in event.obstacles) {
      if (obstacle.zone == ObstacleZone.clear) continue;
      final lastAlert = _lastAlertTime[obstacle.trackingId];
      if (lastAlert != null && now.difference(lastAlert) < _suppressionPeriod) {
        continue; // Suppressed same obstacle, too recent
      }
      _lastAlertTime[obstacle.trackingId] = now;
      final directionStr = obstacle.direction.name; // 'left', 'center', 'right'
      final urgent = obstacle.zone == ObstacleZone.near;
      await _tts.speak(
        '\${obstacle.label} \${directionStr}',
        priority: urgent ? TtsPriority.critical : TtsPriority.high,
        interrupt: urgent,
      );
    }
    emit(ObstacleDetecting(event.obstacles));
  }

  Future<void> _onStop(StopObstacleDetection event, Emitter<ObstacleState> emit)
async {
    await _detectionSubscription?.cancel();
    _lastAlertTime.clear();
    await _tts.speak('Obstacle detection stopped.', priority:
TtsPriority.normal);
    emit(ObstacleIdle());
  }

  @override
  Future<void> close() {
    _detectionSubscription?.cancel();
    return super.close();
  }
}

```



```
OcrBloc(this._repo, this._tts, this._camera) : super(OcrIdle()) {
  on<TriggerOcr>(_onTrigger);
  on<CancelOcrReading>(_onCancel);
}

Future<void> _onTrigger(TriggerOcr event, Emitter<OcrState> emit) async {
  emit(OcrCapturing());
  await _tts.speak('Capturing text.', priority: TtsPriority.high);

  final image = await _camera.takePicture();
  emit(OcrProcessing());

  final result = await _repo.recognizeText(image);
  result.fold(
    (failure) {
      emit(OcrError(failure.message));
      _tts.speak('Text recognition failed. Please try again.',
        priority: TtsPriority.high);
    },
    (ocrResult) {
      if (!ocrResult.success || ocrResult.text.isEmpty) {
        emit(OcrNoText());
        _tts.speak(
          'No text detected. Try moving the camera closer.',
          priority: TtsPriority.high,
        );
        return;
      }
      emit(OcrReading(ocrResult.text));
      _tts.setLanguage(ocrResult.language);
      _tts.speak(ocrResult.text, priority: TtsPriority.normal);
    },
  );
}

Future<void> _onCancel(CancelOcrReading event, Emitter<OcrState> emit) async {
  await _tts.stop();
  emit(OcrIdle());
}
}
```

6. Feature Implementation: Scene Description

6.1 Remote Data Source

```
// lib/features/scene_description/data/datasources/scene_remote_datasource.dart
import 'dart:io';
import 'package:dio/dio.dart';
import 'package:image/image.dart' as img;

class SceneRemoteDatasource {
  final Dio _dio;
  SceneRemoteDatasource(this._dio);

  Future<String> describeScene(File imageFile) async {
    // Compress image to <= 512 KB before sending
    final compressed = await _compressImage(imageFile);

    final formData = FormData.fromMap({
      'image': MultipartFile.fromBytes(compressed, filename: 'scene.jpg'),
    });

    final response = await _dio.post(
      '/scene/describe',
      data: formData,
    ).timeout(const Duration(seconds: 8));

    return response.data['description'] as String;
  }

  Future<List<int>> _compressImage(File file) async {
    final original = img.decodeImage(await file.readAsBytes());!;
    // Resize to max 800px on longest side
    final resized = original.width > original.height
      ? img.copyResize(original, width: 800)
      : img.copyResize(original, height: 800);
    return img.encodeJpg(resized, quality: 75);
  }
}
```

6.2 Scene Description BLoC

```
class SceneBloc extends Bloc<SceneEvent, SceneState> {
  final SceneRepository _repo;
  final ConnectivityService _connectivity;
  final TtsService _tts;
  final CameraController _camera;

  SceneBloc(this._repo, this._connectivity, this._tts, this._camera)
    : super(SceneIdle()) {
    on<RequestSceneDescription>(_onRequest);
  }

  Future<void> _onRequest(
    RequestSceneDescription event,
    Emitter<SceneState> emit,
  ) async {
    final connected = await _connectivity.isConnected;
    if (!connected) {
      emit(SceneOfflineError());
      await _tts.speak(
        'Scene description requires an internet connection. Please try again',
        priority: TtsPriority.high,
      );
      return;
    }
  }
}
```

```
    }

    emit(SceneLoading());
    await _tts.speak('Describing the scene, please wait.', priority:
TtsPriority.high);

    final image = await _camera.takePicture();
    final result = await _repo.describeScene(File(image.path));
    result.fold(
      (failure) {
        emit(SceneError());
        _tts.speak(
          'Scene description is unavailable right now. Please try again later.',
          priority: TtsPriority.high,
        );
      },
      (description) {
        emit(SceneLoaded(description));
        _tts.speak(description, priority: TtsPriority.normal);
      },
    );
  }
}
```


7. Feature Implementation: Currency Detection

```
//
lib/features/currency_detection/data/datasources/tflite_currency_datasource.dart
class TfliteCurrencyDatasource {
  late Interpreter _interpreter;
  static const _inputSize = 224;
  static const _confThreshold = 0.85;
  final List<String> _labels = [
    'fcfa_500', 'fcfa_1000', 'fcfa_2000', 'fcfa_5000', 'fcfa_10000',
  ];
  final Map<String, String> _labelToSpoken = {
    'fcfa_500': 'Five hundred francs CFA',
    'fcfa_1000': 'One thousand francs CFA',
    'fcfa_2000': 'Two thousand francs CFA',
    'fcfa_5000': 'Five thousand francs CFA',
    'fcfa_10000': 'Ten thousand francs CFA',
  };

  Future<void> init() async {
    _interpreter = await Interpreter.fromAsset(
      'assets/models/currency_detection_v1.tflite',
    );
  }

  CurrencyResult classify(img.Image frame) {
    final resized = img.copyResize(frame, width: _inputSize, height:
    _inputSize);
    final input = _normaliseToFloat32(resized); // [-1, 1] normalisation
    final output = List.generate(1, (_) => List.filled(_labels.length, 0.0));
    _interpreter.run(input, output);

    final scores = output[0];
    final maxIdx = scores.indexWhere(
      (s) => s == scores.reduce((a, b) => a > b ? a : b),
    );
    final maxConf = scores[maxIdx];

    if (maxConf < _confThreshold) {
      return CurrencyResult(success: false, confidence: maxConf);
    }
    return CurrencyResult(
      success: true,
      label: _labels[maxIdx],
      spokenLabel: _labelToSpoken[_labels[maxIdx]]!,
      confidence: maxConf,
    );
  }
}
```

8. Voice Command System

NOVA uses the `speech_to_text` plugin for always-on voice command recognition. Commands are matched against a keyword map. The voice system operates in parallel with all feature modules and can interrupt any non-critical TTS output.

```
// lib/core/voice/voice_command_service.dart
import 'package:speech_to_text/speech_to_text.dart';
import 'package:injectable/injectable.dart';

@singleton
class VoiceCommandService {
  final SpeechToText _speech = SpeechToText();
  final VoiceCommandRouter _router;

  // Command keyword map
  static const Map<String, VoiceCommand> _commands = {
    'start obstacle': VoiceCommand.startObstacle,
    'stop obstacle': VoiceCommand.stopObstacle,
    'read text': VoiceCommand.readText,
    'lire texte': VoiceCommand.readText, // French
    'describe scene': VoiceCommand.describeScene,
    'décrire scène': VoiceCommand.describeScene, // French
    'identify money': VoiceCommand.identifyMoney,
    'identifier argent': VoiceCommand.identifyMoney,
    'who is this': VoiceCommand.recogniseFace,
    'qui est': VoiceCommand.recogniseFace,
    'call for help': VoiceCommand.emergency,
    'au secours': VoiceCommand.emergency,
    'slow down': VoiceCommand.slowTts,
    'speed up': VoiceCommand.speedUpTts,
    'stop': VoiceCommand.stopTts,
  };

  Future<void> init(String locale) async {
    await _speech.initialize(
      onError: (err) => _onError(err),
      debugLogging: false,
    );
  }

  void startListening() {
    _speech.listen(
      onResult: (result) {
        if (!result.finalResult) return;
        _processTranscript(result.recognizedWords.toLowerCase());
      },
      listenFor: const Duration(seconds: 30),
      pauseFor: const Duration(seconds: 3),
      partialResults: false,
      localeId: 'en_CM',
    );
  }

  void _processTranscript(String transcript) {
    for (final entry in _commands.entries) {
      if (transcript.contains(entry.key)) {
        _router.dispatch(entry.value);
        return;
      }
    }
  }
}

enum VoiceCommand {
  startObstacle, stopObstacle, readText, describeScene,
```

```
    identifyMoney, recogniseFace, emergency,  
    slowTts, speedUpTts, stopTts,  
  }
```

9. Local Database (Drift/SQLite)

```
// lib/core/database/app_database.dart
import 'package:drift/drift.dart';
import 'package:drift/native.dart';
import 'package:path_provider/path_provider.dart';
import 'package:path/path.dart' as p;

// Table definitions
class UsageEvents extends Table {
  IntColumn get id => integer().autoIncrement();
  TextColumn get moduleId => text().withLength(max: 20);
  DateTimeColumn get timestamp => dateTime();
  TextColumn get outcome => text().withLength(max: 50);
  RealColumn get confidenceScore => real().nullable();
  BoolColumn get synced => boolean().withDefault(const Constant(false));
}

class UserFeedbacks extends Table {
  IntColumn get id => integer().autoIncrement();
  IntColumn get eventId => integer().references(UsageEvents, #id);
  BoolColumn get isPositive => boolean();
  DateTimeColumn get timestamp => dateTime();
  BoolColumn get synced => boolean().withDefault(const Constant(false));
}

class PendingSyncQueue extends Table {
  IntColumn get id => integer().autoIncrement();
  TextColumn get endpoint => text();
  TextColumn get payloadJson => text();
  DateTimeColumn get createdAt => dateTime();
  IntColumn get retryCount => integer().withDefault(const Constant(0));
}

@DriftDatabase(tables: [UsageEvents, UserFeedbacks, PendingSyncQueue])
class AppDatabase extends _AppDatabase {
  AppDatabase() : super(_openConnection());
  @override int get schemaVersion => 1;

  Future<List<UsageEvent>> getUnsynced() =>
    (select(usageEvents)..where((t) => t.synced.equals(false))).get();

  Future<void> markSynced(List<int> ids) =>
    (update(usageEvents)..where((t) => t.id.isIn(ids)))
      .write(const UsageEventsCompanion(synced: Value(true)));
}

LazyDatabase _openConnection() => LazyDatabase(() async {
  final dir = await getApplicationDocumentsDirectory();
  return NativeDatabase.createInBackground(File(p.join(dir.path, 'nova.db')));
});
```

10. Offline Sync Engine

10.1 Overview

When the device has no network, NOVA writes all usage events and feedback to the local SQLite database with `synced = false`. A background `SyncService` watches for connectivity restoration and automatically flushes the local queue to the backend. The sync is idempotent; each event has a UUID generated on the device, so if the network drops mid-batch, re-sending the same batch causes no duplicate inserts on the server.

10.2 SyncService

None

```
// lib/core/sync/sync_service.dart
import 'dart:async';
import 'package:injectable/injectable.dart';
import '../network/connectivity_service.dart';
import '../database/app_database.dart';
import '../network/dio_client.dart';

@singleton class SyncService {
  final ConnectivityService _connectivity;
  final AppDatabase _db;
  final DioClient _dio;
  StreamSubscription? _connectivitySubscription;
  bool _isSyncing = false;
  static const int _maxBatchSize = 50;
  static const int _maxRetries = 3;
  static const Duration _retryBackoffBase = Duration(seconds: 2);

  SyncService(this._connectivity, this._db, this._dio);

  /// Call this once from main.dart after dependency injection is ready.
  void startWatching() {
    _connectivitySubscription = _connectivity.onConnectivityChanged
      .where((isConnected) => isConnected) // only fire on reconnection
      .listen((_) => _triggerSync());
  }

  /// Also call this manually on app foreground resume.
  Future<void> syncNow() => _triggerSync();

  Future<void> _triggerSync() async {
    if (_isSyncing) return; // prevent concurrent sync runs
    _isSyncing = true;
    try {
      await _syncUsageEvents();
      await _syncFeedback();
      await _drainPendingQueue();
    } finally {
      _isSyncing = false;
    }
  }

  Future<void> _syncUsageEvents() async {
    final unsynced = await _db.getUnsyncedEvents();
    if (unsynced.isEmpty) return;
    for (int i = 0; i < unsynced.length; i += _maxBatchSize) {
      final batch = unsynced.skip(i).take(_maxBatchSize).toList();
      final success = await _postWithRetry(
        endpoint: '/logs/sync',
        payload: {
          'events': batch.map((e) => {
            'id': e.serverId,
            'module_id': e.moduleId,
            'timestamp': e.timestamp.toIso8601String(),
            'outcome': e.outcome,
            'confidence_score': e.confidenceScore,
          }).toList(),
        },
      );
      if (success) { await _db.markEventsSynced(batch.map((e) => e.id).toList()); }
    }
  }
}
```

```

Future<void> _syncFeedback() async {
  final unsynced = await _db.getUnsyncedFeedback();
  if (unsynced.isEmpty) return;
  for (int i = 0; i < unsynced.length; i += _maxBatchSize) {
    final batch = unsynced.skip(i).take(_maxBatchSize).toList();
    final success = await _postWithRetry(
      endpoint: '/logs/feedback/sync',
      payload: {
        'feedbacks': batch.map((f) => {
          'id': f.serverId,
          'event_id': f.eventServerId,
          'is_positive': f.isPositive,
          'timestamp': f.timestamp.toIso8601String(),
        }).toList(),
      },
    );
    if (success) { await _db.markFeedbackSynced(batch.map((f) =>
f.id).toList()); }
  }
}

Future<void> _drainPendingQueue() async {
  final pending = await _db.getPendingQueueItems();
  for (final item in pending) {
    if (item.retryCount >= _maxRetries) { await
_db.deletePendingQueueItem(item.id); continue; }
    final success = await _postWithRetry(endpoint: item.endpoint, payload:
item.payloadJson, maxAttempts: 1);
    if (success) { await _db.deletePendingQueueItem(item.id); } else { await
_db.incrementRetryCount(item.id); }
  }
}

Future<bool> _postWithRetry({required String endpoint, required dynamic
payload, int maxAttempts = _maxRetries}) async {
  for (int attempt = 0; attempt < maxAttempts; attempt++) {
    try { await _dio.client.post(endpoint, data: payload); return true; }
    catch (e) { if (attempt < maxAttempts - 1) await
Future.delayed(_retryBackoffBase * (1 << attempt)); }
  } return false;
}

void dispose() { _connectivitySubscription?.cancel(); }
}

```

10.3 Required Drift Query Methods

Add these methods to your AppDatabase class in lib/core/database/app_database.dart:

```

None
// lib/core/database/app_database.dart additions
Future<List<UsageEvent>> getUnsyncedEvents() => (select(usageEvents)
  ..where((t) => t.synced.equals(false))
  ..orderBy([(t) => OrderingTerm.asc(t.timestamp)])) .get();

Future<void> markEventsSynced(List<int> localIds) =>
(update(usageEvents)..where((t) => t.id.isIn(localIds)))
  .write(const UsageEventsCompanion(synced: Value(true)));

Future<List<UserFeedback>> getUnsyncedFeedback() => (select(userFeedbacks)
  ..where((t) => t.synced.equals(false))) .get();

Future<void> markFeedbackSynced(List<int> localIds) =>
(update(userFeedbacks)..where((t) => t.id.isIn(localIds)))

```

```

        .write(const UserFeedbacksCompanion(synced: Value(true)));

Future<List<PendingSyncQueueData>> getPendingQueueItems() =>
(select(pendingSyncQueue)
  ..where((t) => t.retryCount.isSmallerThanValue(_maxRetries))
  ..orderBy([(t) => OrderingTerm.asc(t.createdAt)]))
  .get();

Future<void> deletePendingQueueItem(int id) =>
(delete(pendingSyncQueue)..where((t) => t.id.equals(id))).go();

Future<void> incrementRetryCount(int id) async {
  final item = await (select(pendingSyncQueue)..where((t) =>
t.id.equals(id))).getSingleOrNull();
  if (item == null) return;
  await (update(pendingSyncQueue)..where((t) => t.id.equals(id))
    .write(PendingSyncQueueCompanion(retryCount: Value(item.retryCount + 1))));
}

```

10.4 Wiring SyncService into main.dart

```

None

// Add these two lines after configureDependencies() in main.dart
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await configureDependencies();
  final syncService = getIt<SyncService>();
  syncService.startWatching();
  runApp(const NovaApp());
}

class _HomeMenuPageState extends State<HomeMenuPage> with
WidgetsBindingObserver {
  @override void initState() { super.initState();
WidgetsBinding.instance.addObserver(this); }
  @override void didChangeAppLifecycleState(AppLifecycleState state) {
    if (state == AppLifecycleState.resumed) { getIt<SyncService>().syncNow(); }
  }
  @override void dispose() { WidgetsBinding.instance.removeObserver(this);
super.dispose(); }
}

```

10.5 How the Full Offline-to-Sync Flow Works

Step	What happens
1	User triggers any NOVA feature. Event written to SQLite with synced = false.
2	App checks connectivity. If offline, stops here. Event sits in the local DB.
3	Network returns. ConnectivityService stream emits true.
4	SyncService._triggerSync() fires automatically.
5	Unsynced rows fetched from SQLite, batched into groups of 50.
6	Each batch POSTed to /logs/sync. Server does an idempotent insert (UUID match).
7	On success, rows marked synced = true in SQLite.

Step	What happens
8	On failure, rows stay synced = false. Retried on the next connectivity event.
9	After 3 failed attempts on a pending queue item, it is dropped to prevent indefinite accumulation.

11. Main App Entry Point and Routing

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'injection_container.dart';
import 'features/auth/presentation/pages/onboarding_page.dart';
import 'features/obstacle_detection/presentation/pages/obstacle_page.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await configureDependencies(); // GetIt setup
  runApp(const NovaApp());
}

class NovaApp extends StatelessWidget {
  const NovaApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'NOVA',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        // High-contrast theme for low-vision users
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF1F4E79),
          brightness: Brightness.dark,
        ),
        useMaterial3: true,
      ),
      home: const OnboardingPage(),
      routes: {
        '/home': (_) => const HomeMenuPage(),
        '/obstacle': (_) => const ObstaclePage(),
        '/ocr': (_) => const OcrPage(),
        '/scene': (_) => const ScenePage(),
        '/currency': (_) => const CurrencyPage(),
        '/faces': (_) => const FaceRecognitionPage(),
        '/settings': (_) => const SettingsPage(),
        '/enrolment': (_) => const FaceEnrolmentPage(),
      },
    );
  }
}
```

12. Testing Strategy

Test Type	Tool	Coverage Target	Key Test Cases
Unit tests BLoCs	bloc_test, mocktail	>= 80% of BLoC logic	Alert suppression, offline fallback, empty OCR result, low-confidence currency
Unit tests Use Cases	mocktail	100% of use cases	All success and failure paths
Widget tests Pages	flutter_test	Key interactive elements	TalkBack ContentDescription, button trigger, state transitions

Test Type	Tool	Coverage Target	Key Test Cases
Integration tests TFLite	flutter_test + real device	All 3 on-device models	Inference runs without crash; latency within NFR bounds
E2E tests Full flow	flutter_driver / Patrol	5 core user journeys	Obstacle alert, OCR read, scene describe, currency ID, face recognition